# Zipcode Group Project 2

1.0.0

Generated by Doxygen 1.13.2

# Chapter 1

# Class Index

## 1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

# Chapter 2

# File Index

## 2.1 File List

Here is a list of all documented files with brief descriptions:

# Chapter 3

# Class Documentation

## 3.1 CSVBuffer Class Reference

CSVBuffer reads records from a CSV file sequentially, including the header row.

`#include <CSVBuffer.h>`

### Public Member Functions

- CSVBuffer (const std::string &filePath)
  *Constructor that opens the CSV file and reads the header.*
- ~**CSVBuffer** ()
  *Destructor that closes the file stream.*
- std::vector< std::string > getHeaders () const
  *Retrieves the column headers.*
- bool getNextRecord (std::vector< std::string > &record)
  *Reads the next CSV record into a vector of strings.*
- void **reset** ()
  *Resets the file pointer to the beginning of the file and re-reads the header.*

### 3.1.1 Detailed Description

CSVBuffer reads records from a CSV file sequentially, including the header row.

### 3.1.2 Constructor & Destructor Documentation

#### 3.1.2.1 CSVBuffer()

```
CSVBuffer::CSVBuffer (
            const std::string & filePath)
```

Constructor that opens the CSV file and reads the header.

**Parameters**

| | |
|---|---|
| *filePath* | Path to the CSV file. |

### 3.1.3 Member Function Documentation

#### 3.1.3.1 getHeaders()

```
std::vector< std::string > CSVBuffer::getHeaders () const
```

Retrieves the column headers.

**Returns**

A vector of strings containing the column header names.

#### 3.1.3.2 getNextRecord()

```
bool CSVBuffer::getNextRecord (
            std::vector< std::string > & record)
```

Reads the next CSV record into a vector of strings.

**Parameters**

| | |
|---|---|
| *record* | A vector that will contain the fields of the CSV record. |

**Returns**

True if a record was successfully read, false if end-of-file is reached.

The documentation for this class was generated from the following files:

- CSVBuffer.h
- CSVBuffer.cpp

## 3.2 FieldInfo Struct Reference

Stores information about a single field in a record.

```
#include <FileHeader.h>
```

**Public Attributes**

- std::string fieldName
- std::string typeSchema
- bool isPrimaryKey

### 3.2.1 Detailed Description

Stores information about a single field in a record.

This structure holds the field name, its data type schema, and a flag indicating if this field is the primary key.

### 3.2.2 Member Data Documentation

#### 3.2.2.1 fieldName

```
std::string FieldInfo::fieldName
```

Name or ID of the field

#### 3.2.2.2 isPrimaryKey

```
bool FieldInfo::isPrimaryKey
```

True if this field serves as the primary key

#### 3.2.2.3 typeSchema

```
std::string FieldInfo::typeSchema
```

Data type (e.g., "string", "int")

The documentation for this struct was generated from the following file:

- FileHeader.h

## 3.3 FileHeader Struct Reference

Contains metadata for the file structure.

```
#include <FileHeader.h>
```

#### Public Attributes

- std::string fileStructureType
- std::string version
- size_t headerSize
- size_t recordLengthFieldSize
- std::string sizeFormatType
- std::string primaryKeyIndexFileName
- size_t recordCount
- size_t fieldCount
- std::vector< FieldInfo > fields

### 3.3.1 Detailed Description

Contains metadata for the file structure.

This structure is used to store header information for the structured file format, including file type, version, size parameters, and field definitions.

### 3.3.2 Member Data Documentation

#### 3.3.2.1 fieldCount

```
size_t FileHeader::fieldCount
```

Number of fields per record

#### 3.3.2.2 fields

```
std::vector<FieldInfo> FileHeader::fields
```

Vector containing definitions for each field

#### 3.3.2.3 fileStructureType

```
std::string FileHeader::fileStructureType
```

Type identifier (e.g., "ZIPCODE_DATA")

#### 3.3.2.4 headerSize

```
size_t FileHeader::headerSize
```

Total size in bytes of the header record

#### 3.3.2.5 primaryKeyIndexFileName

```
std::string FileHeader::primaryKeyIndexFileName
```

Name of the primary key index file

#### 3.3.2.6 recordCount

```
size_t FileHeader::recordCount
```

Number of data records in the file

#### 3.3.2.7 recordLengthFieldSize

```
size_t FileHeader::recordLengthFieldSize
```

Number of bytes used for each record length indicator

#### 3.3.2.8 sizeFormatType

```
std::string FileHeader::sizeFormatType
```

Format type for record length ("ASCII" or "binary")

#### 3.3.2.9 version

```
std::string FileHeader::version
```

Version of the file structure (e.g., "1.0")

The documentation for this struct was generated from the following file:

- FileHeader.h

## 3.4 HeaderRecordBuffer Class Reference

The HeaderRecordBuffer class handles reading and writing the header record of the structured (LRF) file.

```
#include <HeaderRecordBuffer.h>
```

### Public Member Functions

- **HeaderRecordBuffer** (const std::string &filePath)
  *Constructor that takes the file path of the LRF file.*
- void **readHeader** ()
  *Reads the header record from the file.*
- void **writeHeader** ()
  *Writes the header record to the file.*
- std::string **getFileType** () const
- std::string **getVersion** () const
- std::string **getRecordLengthFieldSize** () const
- std::string **getSizeFormatType** () const
- std::string **getPrimaryKeyIndexFileName** () const
- std::string **getRecordCount** () const
- std::string **getFieldCount** () const
- std::vector< std::string > **getFieldDefinitions** () const
- void **setFileType** (const std::string &fileType)
- void **setVersion** (const std::string &version)

### 3.4.1 Detailed Description

The HeaderRecordBuffer class handles reading and writing the header record of the structured (LRF) file.

The header record is stored as plain text with each item on a separate line: Line 1: file structure type (e.g., ZIPCODE_DATA) Line 2: version (e.g., 1.0) Line 3: record length field size (e.g., 4) Line 4: size format type (e.g., binary) Line 5: primary key index file name (e.g., primary_index.txt) Line 6: record count (e.g., 40933) Line 7: field count (e.g., 6) Lines 8+: field definitions (each line formatted as: FieldName,TypeSchema,PrimaryKeyIndicator)

The documentation for this class was generated from the following files:

- HeaderRecordBuffer.h
- HeaderRecordBuffer.cpp

## 3.5 LRFBuffer Class Reference

LRFBuffer reads records from a structured file that begins with a header record and has length-indicated records.

`#include <LRFBuffer.h>`

#### Public Member Functions

- LRFBuffer (const std::string &filePath)

  *Constructor that opens the structured file and reads the header.*
- ~**LRFBuffer** ()

  *Destructor that closes the file stream.*
- std::vector< std::string > **getHeaders** () const

  *Returns the column headers (extracted from the header record).*
- bool getNextRecord (std::vector< std::string > &record)

  *Reads the next record from the file.*
- std::streampos **getCurrentFilePosition** ()
- void **seekTo** (std::streampos pos)
- void **reset** ()

  *Resets the file pointer to the beginning and re-reads the header.*

### 3.5.1 Detailed Description

LRFBuffer reads records from a structured file that begins with a header record and has length-indicated records.

The file format is assumed to be:

1. A binary header size (stored as size_t).
2. A header string (in ASCII) of that many bytes. The header string contains:
   - Line 1: fileStructureType
   - Line 2: version
   - Line 3: recordLengthFieldSize
   - Line 4: sizeFormatType
   - Line 5: primaryKeyIndexFileName
   - Line 6: recordCount
   - Line 7: fieldCount
   - Lines 8+: for each field: fieldName,typeSchema,isPrimaryKey
3. For each record:
   - A binary record length (e.g., a 4-byte unsigned int)
   - The record text (a comma-separated string) of that length.

### 3.5.2 Constructor & Destructor Documentation

#### 3.5.2.1 LRFBuffer()

```
LRFBuffer::LRFBuffer (
            const std::string & filePath)
```

Constructor that opens the structured file and reads the header.

**Parameters**

| | |
|---|---|
| *filePath* | Path to the structured file. |

### 3.5.3 Member Function Documentation

#### 3.5.3.1 getNextRecord()

```
bool LRFBuffer::getNextRecord (
            std::vector< std::string > & record)
```

Reads the next record from the file.

**Parameters**

| | |
|---|---|
| *record* | A vector that will contain the fields of the record. |

**Returns**

True if a record was successfully read; false if end-of-file is reached.

The documentation for this class was generated from the following files:

- LRFBuffer.h
- LRFBuffer.cpp

## 3.6 StateExtremes Struct Reference

**Public Member Functions**

- **StateExtremes** (const std::string &zip, double lat, double lon)

**Public Attributes**

- std::string **easternmostZip**
- std::string **westernmostZip**
- std::string **northernmostZip**
- std::string **southernmostZip**
- double **easternmostLon**
- double **westernmostLon**
- double **northernmostLat**
- double **southernmostLat**

The documentation for this struct was generated from the following file:

- StateExtremesReport.cpp

# Chapter 4

# File Documentation

## 4.1 CSVBuffer.h

```
00001 #ifndef CSVBUFFER_H
00002 #define CSVBUFFER_H
00003
00004 #include <fstream>
00005 #include <string>
00006 #include <vector>
00007 #include <sstream>
00008 #include <stdexcept>
00009
00011 class CSVBuffer {
00012 public:
00015     CSVBuffer(const std::string& filePath);
00016
00018     ~CSVBuffer();
00019
00022     std::vector<std::string> getHeaders() const;
00023
00027     bool getNextRecord(std::vector<std::string>& record);
00028
00030     void reset();
00031
00032 private:
00033     std::ifstream inputFile;
00034     std::vector<std::string> headers;
00035 };
00036
00037 #endif // CSVBUFFER_H
```

## 4.2 CSVToLengthIndicated.h

```
00001 #ifndef CSVTOLENGTHINDICATED_H
00002 #define CSVTOLENGTHINDICATED_H
00003
00004 #include <string>
00005
00009 void writeStructuredFile(const std::string& csvFile, const std::string& outputFile);
00010
00011 #endif // CSVTOLENGTHINDICATED_H
00012
```

## 4.3 FileHeader.h File Reference

Defines structures for file header metadata.

```
#include <string>
#include <vector>
```

**Classes**

- struct FieldInfo

  *Stores information about a single field in a record.*
- struct FileHeader

  *Contains metadata for the file structure.*

### 4.3.1 Detailed Description

Defines structures for file header metadata.

This header defines the data structures that store metadata about the file structure used for processing Zip Code records.

## 4.4 FileHeader.h

Go to the documentation of this file.

```
00001
00008
00009 #ifndef FILEHEADER_H
00010 #define FILEHEADER_H
00011
00012 #include <string>
00013 #include <vector>
00014
00022 struct FieldInfo {
00023     std::string fieldName;
00024     std::string typeSchema;
00025     bool isPrimaryKey;
00026 };
00027
00035 struct FileHeader {
00036     std::string fileStructureType;
00037     std::string version;
00038     size_t headerSize;
00039     size_t recordLengthFieldSize;
00040     std::string sizeFormatType;
00041     std::string primaryKeyIndexFileName;
00042     size_t recordCount;
00043     size_t fieldCount;
00044     std::vector<FieldInfo> fields;
00045 };
00046
00047 #endif // FILEHEADER_H
```

## 4.5 HeaderRecordBuffer.h

```
00001 #ifndef HEADERRECORDBUFFER_H
00002 #define HEADERRECORDBUFFER_H
00003
00004 #include <string>
00005 #include <vector>
00006
00021 class HeaderRecordBuffer {
00022 public:
00024     HeaderRecordBuffer(const std::string& filePath);
00025
00027     void readHeader();
00028
00030     void writeHeader();
00031
00032     // Getters for header fields.
00033     std::string getFileType() const;
00034     std::string getVersion() const;
00035     std::string getRecordLengthFieldSize() const;
00036     std::string getSizeFormatType() const;
```

```
00037     std::string getPrimaryKeyIndexFileName() const;
00038     std::string getRecordCount() const;
00039     std::string getFieldCount() const;
00040     std::vector<std::string> getFieldDefinitions() const;
00041 
00042     // Setters for header fields.
00043     void setFileType(const std::string& fileType);
00044     void setVersion(const std::string& version);
00045 
00046 private:
00047     std::string filePath;
00048     // Header fields.
00049     std::string fileType;              // Line 1
00050     std::string version;               // Line 2
00051     std::string recordLengthFieldSize; // Line 3
00052     std::string sizeFormatType;        // Line 4
00053     std::string primaryKeyIndexFileName; // Line 5
00054     std::string recordCount;           // Line 6
00055     std::string fieldCount;            // Line 7
00056     std::vector<std::string> fieldDefinitions; // Lines 8+
00057 };
00058 
00059 #endif // HEADERRECORDBUFFER_H
```

## 4.6 LRFBuffer.h

```
00001 #ifndef LRFBUFFER_H
00002 #define LRFBUFFER_H
00003 
00004 #include <fstream>
00005 #include <string>
00006 #include <vector>
00007 #include <stdexcept>
00008 
00029 class LRFBuffer {
00030 public:
00033     LRFBuffer(const std::string& filePath);
00034 
00036     ~LRFBuffer();
00037 
00039     std::vector<std::string> getHeaders() const;
00040 
00044     bool getNextRecord(std::vector<std::string>& record);
00045 
00046     std::streampos getCurrentFilePosition();
00047 
00048     void seekTo(std::streampos pos);
00049 
00051     void reset();
00052 
00053 private:
00055     void readHeader();
00056 
00057     std::ifstream inputFile;
00058     std::vector<std::string> headers;   // Extracted column headers from the header record.
00059     std::streampos headerEndPosition;   // File position immediately after the header.
00060 };
00061 
00062 #endif // LRFBUFFER_H
```

## 4.7 StateExtremesReport.h

```
00001 #ifndef STATEEXTREMESREPORT_H
00002 #define STATEEXTREMESREPORT_H
00003 
00004 #include <string>
00005 
00016 void runStateExtremesReport(const std::string& lrfFilename);
00017 
00018 #endif // STATEEXTREMESREPORT_H
```